

Aegis CaseOps

Governed Security Incident Remediation on UiPath Maestro

"The LLM is the narrator, never the judge."

Solo: **Hokuto Torigoe** · Built end-to-end with a coding agent (Claude Code)

Security automation fails in two ways: too slow, or too trigger-happy.

- SOC teams drown in alerts. But the scary failure of automation isn't speed – it's an AI isolating a production server on one flaky signal.
- The question we built around: what would it take for a CISO to actually trust an automated incident pipeline?
- Our answer: (1) make the verdict deterministic · (2) keep every evidence gap visible · (3) make it re-auditable by anyone with the raw alert.

Raw alert JSON → a governed incident case on Maestro (BPMN)

alert JSON → API workflow → Maestro BPMN

```
    |
    | [trriage] deterministic core (no LLM)
    |   XOR gateway (4 branches)
low_risk_close / request_more_evidence /
confidence_gap_review / escalate_for_containment
    |
    | queue: caseops-approval (evidence AND gaps)
    |   human approval
remediation-stub → postmortem (gaps included)
```

side channel: Agent Builder + Context Grounding analyst assistant
– explains with citations, cannot write the verdict

Four declared invariants – each with a negative test

- 1** **No remediation without approval** – the approval-required edge is structurally the only path into remediation on the BPMN graph.
- 2** **Gaps propagate, never swallowed** – a failed enrichment tool appears as the same string at intake, approval, and postmortem.
- 3** **Only the deterministic core writes the verdict** – the LLM is read-only, enforced at three layers (prompt/tool, BPMN binding, blind grader).
- 4** **One input schema** – every scenario shares the same alert contract.

Routine alert → auto-close, zero human time

- Two low-severity signals, enrichment all healthy. Risk 2 points, confidence 0.70.
- Corroboration cap: even a 0.9 signal tops out at 0.70 with two signals – a single signal can never auto-escalate.
- Routes straight to `low_risk_close`; the postmortem is already written. "Gaps: none recorded" – the section exists even when empty, so nothing is hidden.

Critical escalation – but remediation stops at the human gate

- New admin role + outbound transfer to an unknown ASN + critical asset. Risk 15 points, 3 correlated signals, confidence 0.76.
- Recommends network isolation + privilege revocation. But invariant #1: **no remediation without approval**.
- The approval payload carries **both evidence and gaps** – the asset-inventory API timeout is shown right where the approver decides.
- The human approves knowing exactly **what the system does not know**.

When tools degrade, the machine gets more careful – not more confident

- 2 of 3 enrichment tools fail. Severity looks high, but confidence collapses to **0.57** and falls below threshold.
- It refuses to auto-escalate and routes to analyst review, recording both tool failures as gaps.
- Degraded input → a more cautious machine. **Partial response applied to the agent topology itself.**

A 60-line blind grader recomputes the verdict from the raw alert

- `verify_verdict.py` takes the raw alert + triage output and diffs the 5 verdict fields.
- Whatever changes a verdict – an LLM, a workflow edit, a hand patch – it **fails loudly with a non-zero exit**.
- Demonstrated: intact → "verdict intact" exit 0 · tampered severity + approval bypass → "VERDICT TAMPERED (2 fields)" exit 1.
- **Judges can try it themselves** – tamper with any verdict field and watch it get caught.

The UiPath SDK as the verdict substrate – not demo decoration

- Three coded agents (trriage / remediation / postmortem) = plain `uipath` SDK, zero LLM in the decision path, `dataclass-in/out`.
- `uipath eval` made real: a custom evaluator + eval set reproduce the verdict contract – all 3 demo scenarios score **1.0**, a tampered expected-set drops one to 0.5.
- Maestro BPMN (XOR gateway + 4 condition edges) · Orchestrator queue · Agent Builder + Context Grounding.
- CLI: `uipath init → run → eval → register → (auth → deploy → invoke)`. Local eval holds even before cloud gates.

Averaging let one 0.9 signal look authoritative. We killed that.

- Corroboration cap: `confidence = min(avg, 0.40 + 0.15 × signal_count)`.
- More independent signals monotonically raise the ceiling. A single signal caps at 0.55 → cannot auto-escalate.
- Severity thresholds 5/9/14 are justified by synthetic boundary tests (4/5, 8/9, 13/14) – not overfit to the demo fixtures.
- **38 pytest cases** pin the 3 demos + 4 invariants (with negative asserts) + threshold boundaries + tamper detection.

Design → adversarial review (blind grader) → TDD – end-to-end in Claude Code

- The design doc was scored leaf-by-leaf by a blind grader → sent back → rewritten until it passed a Definition of Deep.
- Two weaknesses (confidence model, LLM enforcement) were self-discovered and designed out before implementation.
- The commit history is the coding-agent audit trail.

CLOSE

Deterministic verdicts · Visible gaps · Human-gated remediation ·
Independent re-audit

Incident automation a CISO can sign off on.

Built end-to-end with a coding agent.

Repo github.com/Hokutoman00/aegis-caseops-uipath

Track UiPath Maestro BPMN · **Demo video** youtu.be/2XQD_LLDE_o